# Towards a Real-time and Compact Low-light Image Enhancement Method Using Deep Learning on ESP32-S3-EYE

Tho Nguyen[1,2], Long Le[1,2], Hai Nguyen[1,2], Vuong Phuong[1,2], Anh Bui[1,2], Thien Pham[1,2,3], and Tho Quan[1,2,*]

[1] Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), Ho Chi Minh City, Vietnam
[2] Vietnam National University Ho Chi Minh City, Ho Chi Minh City, Vietnam
[3] Ho Chi Minh City University of Foreign Languages – Information Technology (HUFLIT), Vietnam

**Abstract.** Edge computing has emerged as a popular choice for real-time applications thanks to its low-latency, high bandwidth, and data privacy. In surveillance and autonomous systems operating under low-light conditions, effective image enhancement is essential to improve reliability to make decisions based on the quality of images. However, traditional image enhancement methods often fail to recover sufficient details compared to those from deep learning approaches. Despite advantages in performance, deep learning methods usually have difficult in deployment on edge devices due to their high computational and memory consumption. To addressing this gap, we propose a lightweight and efficient low-light image enhancement framework tailored for the ESP32-S3-EYE. Our design incorporates efficient neural network architectures, model compression, and post-training quantization techniques to achieve real-time performance while maintaining high visual quality. This work demonstrates a practical pathway for deploying advanced deep learning-based image enhancement directly on ultra-low-power edge devices. The experimental results demonstrate our promising outcomes, compared to those reported in previous studies.

**Keywords:** ESP32 · Edge AI · low light enhancement · hybrid loss.

## 1 Introduction

Edge computing addresses the limitations of centralized cloud systems by bringing computation closer to end-users, thereby reducing latency and improving robustness [1–4]. This is critical for real-time applications such as surveillance, virtual reality, and traffic monitoring, where even small delays degrade user experience.

At the same time, the surge of mobile multimedia data presents challenges for real-time processing under low-light conditions. Images captured in poor

illumination often suffer from noise and low contrast, which hinder both human perception and vision-based analytics [5, 6]. *Low-light image enhancement* (LLIE) aims to improve brightness, contrast, and detail visibility while minimizing artifacts. Traditional global-based methods often fail to preserve details and cannot meet real-time demands on resource-constrained devices [7]. While deep learning has achieved impressive results, deploying deep neural networks (DNNs) on embedded hardware remains difficult due to compute and memory constraints [8, 9].

In this work, we present a real-time LLIE framework optimized for ESP32-S3-EYE, an ultra-low-power embedded platform. Our design balances enhancement quality, inference speed, and resource usage through:

- **Lightweight Model Design:** Standard convolutions are replaced with depthwise separable operations to reduce parameters and FLOPs.
- **Architecture Optimization:** Filter numbers and operators are carefully minimized to lower computation without significant quality loss.
- **Deployment Techniques:** Quantization compresses the model for real-time execution, while knowledge distillation transfers performance from a larger teacher model to a compact student.

These strategies enable high-quality, real-time LLIE on ESP32 devices, supporting applications such as mobile surveillance and autonomous systems in low-light scenarios without reliance on cloud computing. The remaining parts of the paper are organized as follows. Section 2 presents the research background. Section 3 describes our proposal to address the enhancement on the hardware device. Section 4 reports the experimental results, along with quantitative comparisons. Finally, Section 5 concludes the paper.

## 2 Research Background

### 2.1 ESP32-S3

The ESP32-S3 [11] is a low-power MCU-based system-on-chip (SoC) with integrated 2.4 GHz Wi-Fi and Bluetooth® Low Energy. It incorporates a dual-core Xtensa® 32-bit LX7 processor that enables compact, energy-efficient computation, making it suitable for real-time vision tasks such as low-light image enhancement.

### 2.2 Loss function

**Mean Absolute Error (MAE) Loss** computes the average absolute difference between predicted and ground truth pixel values:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^{N} |x_i - y_i| \tag{1}$$

where $x_i$ and $y_i$ are the predicted and ground truth pixel values, and $N$ is the total number of pixels.

**Structural Similarity Index (SSIM) Loss** is derived from the SSIM metric [10], encouraging perceptual structural similarity rather than pixel-level accuracy:

$$\mathcal{L}_{\text{SSIM}} = 1 - \text{SSIM}(x, y). \tag{2}$$

**Perceptual Loss** measures the Euclidean distance between feature representations extracted from a pretrained network:

$$\mathcal{L}_{\text{percep}} = \sum_{l} \frac{1}{C_l H_l W_l} \|\phi_l(x) - \phi_l(y)\|_2^2 \tag{3}$$

where $\phi_l(\cdot)$ denotes the activation from the $l$-th layer, and $C_l$, $H_l$, and $W_l$ are the number of channels, height, and width of the feature map.

**Charbonnier Loss** is a differentiable variant of L1 loss:

$$\mathcal{L}_{\text{Charb}} = \frac{1}{N} \sum_{i=1}^{N} \sqrt{(x_i - y_i)^2 + \epsilon^2} \tag{4}$$

where $\epsilon$ is a small constant (e.g., $\epsilon = 10^{-3}$).

**Gram Loss** compares the style of two images by measuring the difference between their Gram matrices:

$$\mathcal{L}_{\text{Gram}} = \sum_{l} \|G_l^x - G_l^y\|_F^2 \tag{5}$$

$$G_l^x = \phi_l(x)\phi_l(x)^T, \quad G_l^y = \phi_l(y)\phi_l(y)^T \tag{6}$$

where $\phi_l(x) \in \mathbb{R}^{C_l \times (H_l W_l)}$ is the reshaped feature map from the $l$-th layer, and $\|\cdot\|_F$ is the Frobenius norm.

### 2.3 MobileNet and GhostNet

MobileNet is a lightweight convolutional neural network architecture designed for efficient execution on mobile and embedded devices. Its core innovation is the use of depthwise separable convolutions, which factorize a standard convolution into a depthwise and a pointwise convolution, significantly reducing computational cost:

$$\text{FLOPs}_{\text{standard}} = D_k^2 \cdot M \cdot N \cdot D_f^2 \tag{7}$$

$$\text{FLOPs}_{\text{MobileNet}} = D_k^2 \cdot M \cdot D_f^2 + M \cdot N \cdot D_f^2 \tag{8}$$

where $D_k$ is the kernel size, $D_f$ is the spatial dimension of the feature map, $M$ is the number of input channels, and $N$ is the number of output channels. MobileNet offers a good trade-off between accuracy and computational efficiency [9].

GhostNet introduces Ghost modules to reduce redundancy in convolutional features by generating more feature maps using cheap operations:

$$\mathbf{Y} = [\mathbf{Y}_1, \text{CheapOp}(\mathbf{Y}_1)] \tag{9}$$

Instead of using expensive convolutions to produce all feature maps, only a small portion is generated with standard convolution, and the rest with linear transformations. This drastically reduces computational overhead while maintaining accuracy, making GhostNet suitable for edge AI deployment [12].

### 2.4 Knowledge distillation

Knowledge distillation transfers knowledge from a high-capacity teacher model to a lightweight student model by minimizing a combined loss:

$$\mathcal{L}_{\text{KD}} = \alpha \cdot \mathcal{L}_{\text{CE}}(y, s) + (1 - \alpha) \cdot T^2 \cdot \mathcal{L}_{\text{KL}}(t_T, s_T) \tag{10}$$

where $s$ and $y$ are the student prediction and ground truth, $t_T$ and $s_T$ are teacher and student soft predictions at temperature $T$, and $\alpha$ controls the contribution of each term. This method helps smaller models generalize better and match the performance of larger networks [13].

### 2.5 Post training quantization (PTQ)

*Post-training quantization* (PTQ) is a widely adopted technique to compress deep learning models by reducing the precision of weights and activations—typically from 32-bit floating point (FP32) to 8-bit integer (INT8)—without requiring full retraining. PTQ significantly reduces model size and computational complexity, making it especially suitable for deployment on resource-constrained devices.

### 2.6 Evaluation Metrics

**Structural Similarity Index (SSIM)** SSIM [10] evaluates the perceptual similarity between an enhanced image $x$ and its reference $y$ by comparing luminance, contrast, and structure:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \tag{11}$$

where $\mu_x$, $\mu_y$ are local means, $\sigma_x^2$, $\sigma_y^2$ are variances, and $\sigma_{xy}$ is the covariance. $C_1$ and $C_2$ are stability constants. Higher SSIM values indicate better perceptual quality.

**Peak Signal-to-Noise Ratio (PSNR)** PSNR [10]is a widely used image quality metric for measuring the fidelity of a reconstructed or enhanced image relative to a reference (ground truth). It is defined via the Mean Squared Error (MSE) between the images:

$$\text{MSE}(x, y) = \frac{1}{MN} \sum_{i=1}^{M} \sum_{j=1}^{N} \big(x_{ij} - y_{ij}\big)^2 \tag{12}$$

$$\mathrm{PSNR}(x, y) = 10 \cdot \log_{10} \left( \frac{\mathrm{MAX}_I^2}{\mathrm{MSE}(x, y)} \right) \tag{13}$$

Here, $\mathrm{MAX}_I$ is the maximum possible pixel value of the image. Higher PSNR values indicate smaller reconstruction error and thus better fidelity.

## 3 Our Proposal

Before presenting details, we provide an overview to clarify what is inherited from prior work and what is newly introduced in our framework. Our design inherits well-established components such as depthwise separable convolutions [9], ghost feature generation [12], and the use of ReLU6 (clipped ReLU in [0,6]) to ensure quantization-friendliness. Building upon these foundations, our contributions are threefold: (1) a novel Ghost-Sep residual block with a fusion layout and channel–spatial mixing order specifically tuned for microcontroller constraints, (2) a hybrid loss with automated weight optimization, and (3) a deployment-oriented training pipeline that integrates knowledge distillation with post-training quantization (PTQ). Together, these innovations enable a compact model that achieves high-quality, real-time low-light enhancement on the ESP32-S3.

### 3.1 Progressive Residual Block Design Toward ESP32 Deployment

Deep convolutional neural networks (CNNs) with residual connections enable efficient training of deep vision models [14], but their heavy compute and memory requirements hinder deployment on ultra-low-power devices such as the ESP32-S3, which lacks a GPU and provides only 512KB of on-chip SRAM.

To address this, MobileNet [9] introduced depthwise separable convolutions, reducing parameters and FLOPs in residual blocks. GhostNet [12] further cut complexity by generating additional "ghost" features with lightweight operations, but its design still targets more capable hardware than microcontrollers.

We therefore propose the **ESP32-Optimized Ghost Residual Block**, retaining GhostNet's efficiency principles while adapting for resource-constrained MCUs:

- **Minimalist structure:** 1×1 Conv2D for intrinsic features, 3×3 depthwise convolution for ghost features, and 1×1 Conv2D projection with skip connections for stable gradients.
- **Quantization-aware:** BatchNorm layers are fused into convolutions during INT8 conversion, while ReLU6 activation improves numerical stability and prevents large activations.
- **SRAM efficiency:** Optimized tensor usage and cache locality leverage ESP32's instruction set for faster depthwise operations.

Experiments show that the proposed block maintains strong accuracy across deeper and wider variants while remaining efficient for ESP32 deployment.

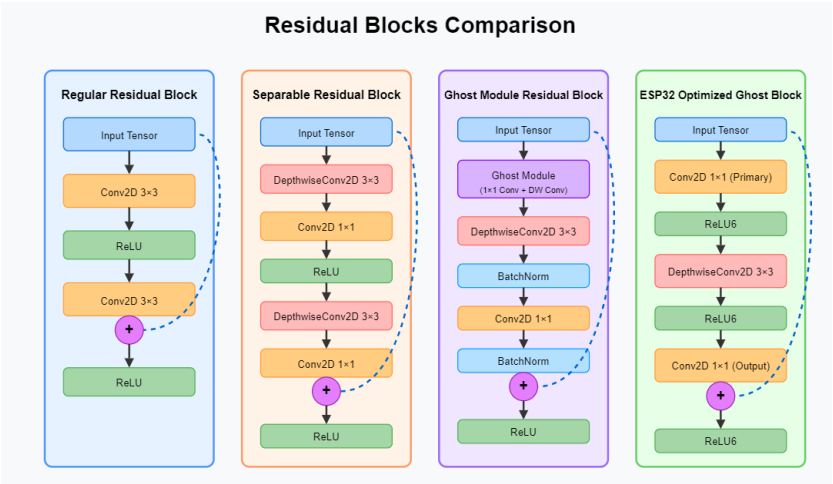


Fig. 1: Comparison of different residual blocks: Regular, Separable, Ghost Module, and the proposed ESP32-optimized Ghost Block.

### 3.2 Hybrid loss

The hybrid loss $\mathcal{L}_{\text{hybrid}}$ combines multiple complementary objectives to guide image reconstruction:

$$\mathcal{L}_{\text{hybrid}} = w_1 \cdot \mathcal{L}_{\text{Charbonnier}} + w_2 \cdot \mathcal{L}_{\text{SSIM}} + w_3 \cdot \mathcal{L}_{\text{Perceptual}} + w_4 \cdot \mathcal{L}_{\text{Gram}} \tag{14}$$

where $w_1$, $w_2$, $w_3$, and $w_4$ are the respective weights of each loss term. $\mathcal{L}_{\text{Charbonnier}}$ captures pixel-level fidelity, $\mathcal{L}_{\text{SSIM}}$ enforces structural similarity, $\mathcal{L}_{\text{Perceptual}}$ aligns deep feature representations from a pretrained network, and $\mathcal{L}_{\text{Gram}}$ encourages stylistic consistency via Gram matrix statistics.

To balance the influence of individual loss components, each metric is first normalized to a common scale based on its value range. We then apply Optuna [15] to optimize the weighting of components by maximizing a combined objective (PSNR + SSIM), as illustrated in Fig. 2.

Due to the strict memory and compute constraints of the ESP32, the deployed model must be compact. To counter accuracy loss from compression, we apply knowledge distillation, enabling a lightweight student model to learn from a high-capacity teacher. The total distillation loss is defined as:

$$\mathcal{L}_{\text{distill}} = (1-\alpha)\cdot\mathcal{L}_{\text{hybrid}} + \alpha\cdot\text{KL}\left(\text{softmax}\left(\frac{y_{\text{teacher}}}{T}\right) \middle\| \text{softmax}\left(\frac{y_{\text{student}}}{T}\right)\right) \tag{15}$$

where $\alpha$ balances the supervised and distillation objectives, $T$ is the temperature, and KL denotes the Kullback–Leibler divergence between softened teacher and student outputs. This approach effectively improves student performance while preserving efficiency.

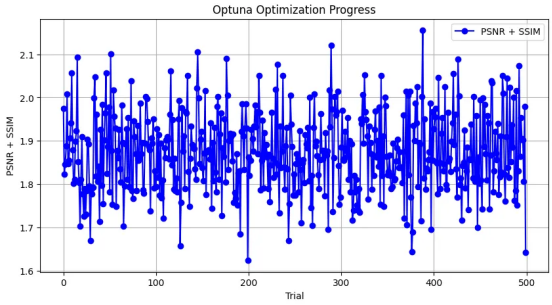


Fig. 2: Optuna optimization progress of hybrid loss weights using the combined objective (PSNR + SSIM), where both metrics are normalized to a common scale.

### 3.3 Model Compression and Deployment

To enable real-time inference on resource-constrained edge devices such as the ESP32-S3, the trained model is first converted to the *TensorFlow Lite* (TFLite) format and further compressed using PTQ, reducing precision from *float32* to *int8*. This step notably decreases model size and enhances inference speed while preserving sufficient accuracy for embedded applications. Deployment is carried out via the *TensorFlow Lite for Microcontrollers* (TFLite Micro) framework.

## 4 Experimental Results

### 4.1 Dataset

We evaluate our method on the LOL dataset [6] and the LOL-v2 dataset [16], which are widely used benchmarks for low-light image enhancement. The original LOL dataset contains 500 paired low/normal-light images for training and 15 pairs for testing. The LOL-v2 dataset extends this benchmark with higher-quality and more diverse scenes, including both real-captured and synthetic subsets. Following standard practice, we train on the training split of each dataset and evaluate on the corresponding test split. All metrics are reported on the RGB images.

### 4.2 Loss Function

We compare two models with identical architectures, trained with either hybrid loss or standard MAE. As shown in Fig. 3, the hybrid-loss model (solid lines) converges smoothly and generalizes better, as indicated by stable validation curves. It consistently outperforms the MAE baseline (dashed lines) in SSIM and PSNR, with smaller train–val gaps and improved reconstruction quality. These results confirm the hybrid loss enhances both training stability and perceptual performance.

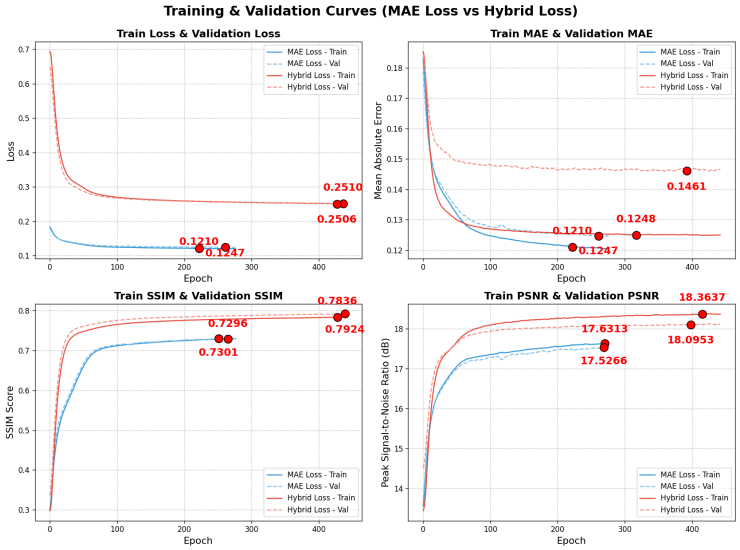


Fig. 3: Training and validation curves comparison between MAE Loss and Hybrid Loss on the LOL/LOLv2 datasets. "Loss" refers to the proposed hybrid objective (Charbonnier + SSIM + Perceptual + Gram), while "MAE" denotes the standard pixel-level loss. Evaluation metrics include MAE, SSIM, and PSN.

### 4.3 Model Architecture

**Training performance comparison** To assess the impact of different residual block architectures, we evaluate four models with identical topologies but distinct convolutional block designs: **Conv2D**, **Separable**, **GhostSeparable**, and **GhostSeparable-ESP**. Their training performance is summarized in Fig. 4.

All models were trained under the same experimental setup: input resolution of $96 \times 96$, Adam optimizer with a learning rate of $10^{-4}$, and early stopping with patience of 15 epochs. Each model consists of two residual blocks with 8 filters per layer, differing only in the block design.

The **Separable** model converges fastest (227 epochs) and achieves the best overall accuracy, with the lowest Loss (0.2486), MAE (0.1159), and highest SSIM (0.7863) and PSNR (18.7425 dB). The **GhostSeparable** model offers a strong trade-off with slightly lower SSIM (0.7841) and PSNR (18.4175 dB), but a longer convergence time (442 epochs), suggesting more complex feature interactions. In contrast, the baseline **Conv2D** model converges at 341 epochs with moderate results (MAE = 0.1258, SSIM = 0.7791, PSNR = 18.5086), but suffers from higher computational cost. Notably, the **GhostSeparable-ESP** model demonstrates the most compact configuration with a reasonable accuracy profile (SSIM = 0.7775, PSNR = 18.3040) and moderate training time (359 epochs), making it well-suited for deployment under resource constraints.

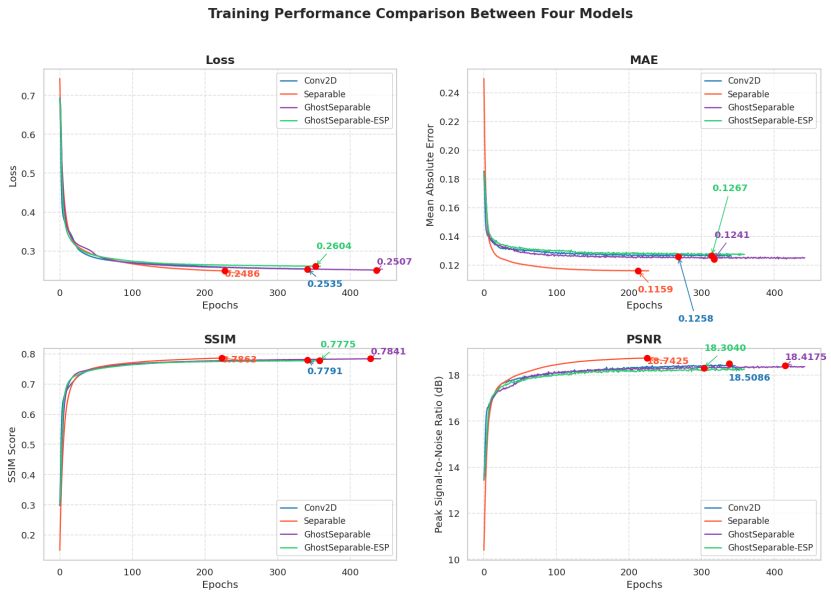


Fig. 4: Training performance comparison of four residual block variants (Conv2D, Separable, GhostSeparable, and GhostSeparable-ESP) on the LOL dataset, evaluated by Loss, MAE, SSIM, and PSNR.

The evaluation shows that lightweight designs—particularly those using depthwise separable and ghost-based operations—can substantially reduce complexity without sacrificing much accuracy. Among them, the GhostSeparable-ESP block provides the best trade-off between model compactness and performance, validating its practicality for real-time edge deployment.

**Evaluation of image enhancement methods** Table 1 summarizes the performance of convolutional block variants evaluated on the LOL and LOLv2 [16] datasets across two resolutions. At 96×96, the **Ghost Separable** block consistently outperforms others, achieving the highest PSNR and SSIM on both datasets. Its optimized variant, **Ghost Sep. (Opt. ESP)**, significantly reduces parameters (from 859 to 291) with minimal quality loss. Applying distillation further enhances performance, validating knowledge transfer as an effective compression strategy.

Scaling to deeper configurations (16 layers, 32 filters) yields consistent improvements, where Ghost Separable maintains top results (SSIM = 0.877 on LOL, PSNR = 19.24 on LOLv2). At higher resolution (256×256), although all models show SSIM degradation, Ghost Separable still leads (SSIM = 0.826), while the optimized ESP version offers a strong accuracy-efficiency balance with just 19.9K parameters.

Finally, distilling a large Ghost Separable model into a compact **Opt. ESP** student significantly improves its reconstruction quality without increasing model

Table 1: Quantitative performance on LOL and LOLv2 at two input resolutions.

| **Model** | **#Params** | **LOL** | | **LOLv2** | |
|---|---|---|---|---|---|
| | | PSNR↑ | SSIM↑ | PSNR↑ | SSIM↑ |
| *96×96 (2 layers, 8 filters)* | | | | | |
| Conv2d-MAE | 2.6K | 16.77 | 0.719 | 16.30 | 0.690 |
| Conv2d | 2.6K | 19.18 | 0.857 | <u>18.36</u> | <u>0.834</u> |
| Separable | 859 | 19.06 | 0.849 | 18.07 | 0.825 |
| GhostSep | 851 | **19.20** | **0.864** | **18.71** | **0.852** |
| Ghost-ESP | 291 | 19.18 | 0.852 | 18.02 | 0.829 |
| Ghost-ESP+Distill | 291 | <u>19.19</u> | <u>0.859</u> | 18.12 | 0.831 |
| *96×96 (16 layers, 32 filters)* | | | | | |
| GhostSep | 38.6K | **19.30** | **0.877** | **19.24** | **0.861** |
| Ghost-ESP | 19.9K | 19.29 | 0.862 | 18.80 | 0.844 |
| *256×256 (16 layers, 32 filters)* | | | | | |
| Conv2d | 291.9K | <u>18.18</u> | <u>0.688</u> | <u>17.46</u> | <u>0.629</u> |
| Separable | 45.0K | 18.10 | 0.683 | 17.26 | 0.625 |
| GhostSep | 38.6K | **19.10** | **0.847** | **18.52** | **0.826** |
| Ghost-ESP | 19.9K | 18.09 | <u>0.688</u> | 17.11 | 0.627 |

size. Overall, Ghost-based designs deliver competitive results at low resolution and low parameter count, making them well-suited for real-time low-light enhancement on resource-constrained devices.

Figure 5 compares our learning-based baselines (Conv2D, SeparableConv2D, Ghost-Separable) with traditional enhancement methods (Gamma correction, Histogram Equalization, CLAHE, Log, Retinex). The proposed model achieves superior visual quality by restoring fine details and preserving color fidelity, while avoiding over-enhancement and contrast degradation, resulting in natural and perceptually pleasing low-light images.

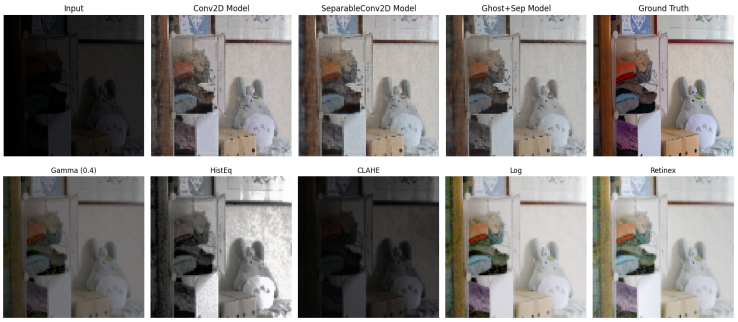


Fig. 5: Comparison of low-light image enhancement results between learning-based models and traditional methods.

### 4.4 Deployment Evaluation on ESP32

We benchmarked four models to assess deployment feasibility. Table 2 reports their model size (FP32/INT8), code footprint, and inference time. Among the benchmarks, Conv2D has the largest footprint and slowest inference (375 ms), limiting embedded deployment. Separable convolutions improve efficiency with reduced latency (289 ms), while the Ghost model, despite its lightweight design, incurs extra overhead from redundant channel expansion (355 ms). In contrast, Ghost-ESP achieves the smallest INT8 size (7.6 KB), lowest code footprint (47.3 KB), and fastest inference (277 ms), offering the best trade-off between efficiency and enhancement quality for real-time deployment on ESP32-S3-EYE.

Table 2: Deployment performance of different models on ESP32-S3.

| **Model** | **FP32 (KB)** | **INT8 (KB)** | **Code Size (data.cc)** | **Inference Time (ms)** |
|---|---|---|---|---|
| Conv2D | 13.5 | 8.8 | 52.8 | 375 |
| Separable | 9.2 | 10.1 | 62.1 | 289 |
| Ghost | 9.7 | 10.9 | 65.7 | 355 |
| Ghost-ESP | **6.2** | **7.6** | **47.3** | **277** |

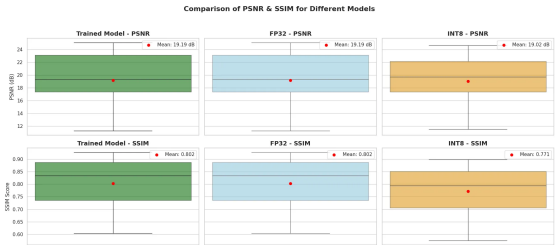


Fig. 6: Box plots of PSNR and SSIM for the original, TFLite FP32, and INT8 quantized models on the LOL dataset. Red dots indicate mean values.

## 5 Conclusion and Outlook

In this paper, we proposed an efficient deep learning model optimized for deployment on low-resource devices such as the ESP32. Through extensive experiments, we demonstrated the advantages of architectural choices and training strategies in improving both perceptual and numerical performance.

The use of a hybrid loss function proved to be particularly effective during training, enabling the model to achieve superior results in PSNR (18.74 dB) and SSIM (0.786) compared to the baseline trained solely with MAE loss. Furthermore, comparative evaluations across different model types (e.g., Conv2D, Separable, Teacher, and Distillation) highlighted the effectiveness of knowledge distillation, with the distilled model achieving the highest quality results (PSNR: 19.20 dB, SSIM: 0.932).

To ensure deployability, we converted the trained model to TFLite format and applied post-training INT8 quantization. Evaluation of the quantized model revealed only a marginal performance drop (PSNR: 19.02 dB, SSIM: 0.771), affirming its suitability for deployment on microcontrollers like ESP32 with minimal loss in quality.

## References


1. W. Shi and S. Dustdar, "The promise of edge computing," *Computer*, vol. 49, no. 5, pp. 78–81, May 2016.
2. M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, Jan. 2017.
3. M. Chiang and T. Zhang, "Fog and IoT: An overview of research opportunities," *IEEE Internet of Things Journal*, vol. 3, no. 6, pp. 854–864, Dec. 2016.
4. Y. Mao, C. You, J. Zhang, et al., "A survey on mobile edge computing: The communication perspective," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, Oct. 2017.
5. K. G. Lore, A. Akintayo, and S. Sarkar, "LLNet: A deep autoencoder approach to natural low-light image enhancement," *Pattern Recognition*, vol. 61, pp. 650–662, Mar. 2017.
6. C. Wei, W. Wang, W. Yang, et al., "Deep retinex decomposition for low-light enhancement," *Proceedings of BMVC*, p. 31, Sep. 2018.
7. R. Zhang, Y. Zhang, B. Zhong, et al., "Learning to enhance low-light image via zero-reference deep curve estimation," *Proceedings of CVPR*, pp. 3123–3132, Jun. 2019.
8. S. Han, et al., "Learning both weights and connections for efficient neural networks," *Advances in Neural Information Processing Systems*, vol. 28, pp. 1135–1143, Dec. 2015.
9. A. G. Howard, M. Zhu, B. Chen, et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, Apr. 2017.
10. Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004. doi:10.1109/TIP.2003.819861
11. Espressif Systems, "ESP32-S3-EYE Getting Started Guide," available: https://github.com/espressif/esp-who/, accessed Dec. 13, 2024.
12. K. Han *et al.*, "GhostNet: More features from cheap operations," in *Proc. CVPR*, 2020, pp. 1580–1589.
13. G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.
14. K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 770–778, Jun. 2016.
15. T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," *arXiv preprint arXiv:1907.10902*, Jul. 2019. [Online]. Available: https://arxiv.org/abs/1907.10902
16. W. Yang, S. Wang, Y. Fang, Y. Wang, and J. Liu, "From fidelity to perceptual quality: A semi-supervised approach for low-light image enhancement," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 3060–3069, 2020, doi: 10.1109/CVPR42600.2020.00313.
17. J. Guo, J. Ma, Á. F. García-Fernández, Y. Zhang, and H. Liang, "A survey on image enhancement for low-light images," *Heliyon*, vol. 9, no. 4, p. e14558, 2023. [Online]. Available: https://doi.org/10.1016/j.heliyon.2023.e14558